State-Filtered Deliberation: Stage-Isolated Context Management for Small Open-Weight Software Engineering Agents

Karandeep Singh Sodhi
Independent Researcher (`https://github.com/KarandeepSinghSodhi/gemma-sfd-agent`)
Submitted to the Gemma 4 Developer Agent Challenge (Paper Track) — October 2026

Abstract

On the Gemma 4 Developer Agent Challenge Small Model Prize leaderboard (197 teams, 2026-10-24 UTC), public resolution rates on 4-bit quantized E2B/E4B models range from 0.0% to 10.0% (median 5.0%), compared to 22.0%–24.0% for 31B models. We study **State-Filtered Deliberation (SFD)**, a declarative three-stage `SequentialAgent` architecture in Google ADK and `swegemma` (`TriageAgent` → `PatchAgent` → `VerifierAgent`) that configures `include_contents: "none"` on `PatchAgent` and `VerifierAgent` to slice out prior tool observations at stage boundaries while passing structured state variables (`localization_packet` and `patch_manifest`). Across $n = 16$ paired tasks on `gemma-4-e4b-it-qat-w4a16-ct` (64 total runs on 4x NVIDIA L4 GPUs) against the official starter kit (`flat_starter`) and a controlled single-agent baseline (`flat_controlled`), SFD resets stage-entry prompts from a mean `TriageAgent` peak of 10922.1 tokens to 1500.9 tokens at `PatchAgent` entry (-86.3%), lowers per-task peak prompt length relative to `flat_controlled` on 13/16 tasks (sign test $p = 0.0213$, mean 11291.9 vs. 13550.8 tokens; 13029.3 on `flat_starter`, without bounding upper-tail peaks: max 16146 vs. 15281 and 16154), and reduces cumulative total tokens by 25.7% vs. `flat_controlled` (88909.5 vs. 119664.4, sign test $p = 0.0213$). In this 16-task sample, `flat_starter` also bypasses its discretionary `code_analyzer` sub-agent on 13/16 tasks (81.2%). End-to-end task resolution shows a null difference: 1/16 (6.2%, 95% Wilson CI [1.1%, 28.3%]) for `sfd` and `flat_starter`, and 0/16 (0.0%, CI [0.0%, 19.4%]) for `flat_controlled` (McNemar $p = 1.0000$). Code and all 64 traces: `https://github.com/KarandeepSinghSodhi/gemma-sfd-agent`.

1. Introduction & Motivating Hypotheses

Repository-level software engineering benchmarks such as SWE-bench ((Jimenez et al., 2024)) evaluate whether language model agents can diagnose bug reports, navigate multi-file codebases, synthesize patches, and pass unit test suites. While frontier models and 31B open-weight models achieve strong resolution rates using iterative ReAct loops ((Yao et al., 2023); (Yang et al., 2024); (Wang et al., 2024)), deploying autonomous coding agents on local developer workstations requires compact, quantized open-weight models such as the Gemma 4 family (E2B, E4B, 12B, 31B) ((Gemma Team, Google DeepMind, 2026)).

Empirical data from the Google Gemma 4 Developer Agent Challenge highlights the severity of this capability gap. In our archived leaderboard snapshots (`results/leaderboard_snapshot_2026_10_26.json`, retrieved 2026-10-26 UTC, and `results/small_model_leaderboard_2026_10_24.csv` from `ryanholbrook/gemma-4-developer-agent-small-model-leaderboard`, updated 2026-10-24 23:58 UTC), the official competition starter baseline running on `gemma-4-31b-it-qat-w4a16-ct` with fine-tuned LoRA adapters achieves a 22.0% (`0.22`) public resolution rate and the top 31B submissions reach 24.0% (`0.24`). By contrast, across all $N = 197$ participating teams on the official Small Model Prize leaderboard (restricted to E2B/E4B models), public resolution scores range from 0.0% (`0.00`) to 10.0% (`0.10`), with a median of 5.0% (`0.05`), a mean of 4.23% (`0.0423`), and 183 of 197 teams (92.9%) scoring $\leq 6.0\%$ (`0.06`). (Our own full-set competition submission, Ref `#57643078` submitted 2026-10-25 21:22:35 UTC, remained in `SubmissionStatus.PENDING` (`public_score: null`) in the asynchronous evaluation queue at the time of snapshot retrieval on 2026-10-26 UTC.)

We investigate how agent workflow structure and stage-boundary context slicing affect `gemma-4-e4b-it-qat-w4a16-ct` under the competition's declarative Google Agent Development Kit (ADK) and `swegemma` harness. We formulate and test three specific hypotheses:

1. **Prompt-Length Error Correlation Hypothesis (Hypothesis 1):** In multi-turn trajectories, as per-call prompt length P_t increases with accumulated tool observations, we hypothesize that non-budget tool call errors (such as malformed `edit_file` search blocks) and duplicate file reads occur more frequently. We test this observationally by bucketing all recorded LLM calls by prompt token length P_t and separating genuine tool errors from budget-cap rejections.
2. **Discretionary Sub-Agent Under-Utilization Hypothesis (Hypothesis 2):** The official competition starter kit (`sample_submission/agent.yaml`) already provides a read-only `code_analyzer` sub-agent wrapped as an `AgentTool` (`skip_summarization: true`) alongside all nine built-in `swegemma` tools on the root `swe_baseline_agent`. Because invoking `code_analyzer` is discretionary, we hypothesize that a 4B model rarely delegates to `code_analyzer` and instead executes exploratory file reads directly inside its root context window.
3. **Stage-Entry Context Reset & Role Partitioning Hypothesis (Hypothesis 3):** Replacing discretionary delegation with a mandatory `SequentialAgent` pipeline (`TriageAgent` → `PatchAgent` → `VerifierAgent`) that sets `include_contents: "none"`

on `PatchAgent` and `VerifierAgent` deterministically bounds **stage-entry** prompt length and lowers typical per-task peak prompt length (though not upper-tail `TriageAgent` peaks) while separating localization from code editing.

2. Context Mechanics in `swegemma` and Google ADK

2.1 Prompt Growth and the Token Signal Ratio

Consider an agent trajectory across LLM calls $t \in \{1, 2, ..., T\}$. Let P_t denote the prompt token count and C_t the completion token count on call t. In a continuous single-agent session (`include_contents: "default"`), the prompt at call t concatenates the system instruction s_0, the initial harness task prompt u_0, and all prior assistant turns a_i and tool observations o_i:

$$P_t = |s_0| + |u_0| + \sum_{i=1}^{t-1}(|a_i| + |o_i|)$$

In the `swegemma` harness, even at call $t = 1$, $|s_0| + |u_0|$ spans 1,900–4,800 tokens because `build_agent_prompt()` embeds the problem statement, budget rules, tool schemas, and up to 200 breadth-first directory tree entries under `/workspace`. Furthermore, `swegemma` configures Google ADK's `EventsCompactionConfig` (`compaction_interval=15`, `token_threshold=14336`, `event_retention_size=5`) to compact long histories after 15 turns.

2.2 Exact Stage-Boundary Slicing via `include_contents: "none"` in Google ADK

Inspection of `google-adk==1.36.1` (`google/adk/flows/llm_flows/contents.py` and `functions.py`) establishes the exact runtime behavior when `SequentialAgent` transitions between sub-agents:

- When a sub-agent executes a tool call, `functions.py` records both the `function_call` event and the resulting `function_response` event with `event.author = invocation_context.agent.name`.
- When `include_contents: "none"` is configured on `PatchAgent` (or `VerifierAgent`), `_ContentLlmRequestProcessor` calls `_get_current_turn_contents(branch, session.events, agent.name)`. This function scans `session.events` backward from `len(events) - 1` and stops at the first event where `event.author == "user"` or `_is_other_agent_reply(agent.name, event)` is `True`—which is the **immediately preceding sub-agent's final text emission event** e_k.
- `_get_current_turn_contents` slices `session.events[k:]`, excluding `events[:k]` (both `events[0]`, the initial 200-entry workspace layout tree, and all intermediate `function_call`, `function_response`, and thought blocks from earlier stages).
- Crucially, within the active sub-agent's own multi-step loop, all new tool calls and responses (`events[k+1:]`) have `event.author == agent.name`, so the backward scan retains the sub-agent's complete within-stage tool history. Thus, `include_contents: "none"` bounds **stage-entry** prompt length by construction, but does **not** bound within-stage prompt growth if a sub-agent makes many tool calls.

3. Architecture & Runtime-Enforced vs. Prompt-Requested Rules

All configurations execute inside `swegemma`, which provides nine built-in tools: `run_command`, `read_file`, `edit_file`, `write_file`, `get_status`, `submit_patch`, and the pre-computed NetworkX AST call-graph and `.npz` symbol-embedding tools `search_similar_code`, `get_code_neighbors`, and `get_code_subgraph`. SFD (`sub_sfd`) chains three sub-agents under a root `SequentialAgent`:

1. **`TriageAgent` (`include_contents: "default"`, `output_key: "localization_packet"`):** Restricted in YAML to five read-only tools (`search_similar_code`, `get_code_neighbors`, `get_code_subgraph`, `read_file`, `get_status`).
2. **`PatchAgent` (`include_contents: "none"`, `output_key: "patch_manifest"`):** Restricted in YAML to four workspace tools (`read_file`, `edit_file`, `write_file`, `get_status`). Interpolates `{problem_description?}` and `{localization_packet?}` via `inject_session_state()`.
3. **`VerifierAgent` (`include_contents: "none"`):** Configured with `run_command`, `read_file`, `edit_file`, `get_status`, and `submit_patch`. Interpolates `{problem_description?}`, `{localization_packet?}`, and `{patch_manifest?}`.

Table 1 separates constraints enforced in code/YAML from behaviors requested only via prompts. All nine files in `agent_sfd/` (`agent.yaml`, `eval_config.yaml`, `configs/sampling.yaml`, `prompts/*.md`, and `sub_agents/*.yaml`) are byte-for-byte identical to the competition archive `submission_sfd.zip` (SHA-256 `c4857f8d163fe7d64d74b22370ee26ad8f6e23af50678edc4414fb2f2d928124`, submitted to Kaggle as Ref `#57643078` at `2026-10-25 21:22:35 UTC` and committed 52 seconds later in git commit `af0937d4109f956d7d11aa560809bcb4f6bab471` at `21:23:27 UTC`, with zero subsequent commits or untracked edits touching `agent_sfd/`), and the Kaggle evaluation kernels (`kaggle_kernel/`) wrote these exact files from a clean working tree.

Mechanism / Rule	Enforced in Code / YAML Config	Prompt-Requested Only (Unenforced)
Stage execution & tool separation	`SequentialAgent` order; per-stage `tools:` lists in `sub_agents/*.yaml`	Stopping after ≤ 3 tool calls in `TriageAgent` / `PatchAgent`
Cross-stage context reset	`include_contents: "none"` slices `events[k:]` at stage entry	Within-stage prompt growth & per-stage tool call count
State packet handoff	`output_key` writes final text to `session.state`; `{var?}` interpolation	Formatting text inside `<localization_packet>` / `<patch_manifest>` tags
Observation size caps	`read_file` ≤ 150 lines / 10k chars; `run_command` stdout ≤ 5k chars	Reading narrow `start_line..end_line` slices
Test & workspace hygiene	`strip_protected_files_from_patch` strips edits to `tests/*`, `pytest.ini`, `conftest.py`	Running targeted `pytest` (not bare `pytest`); putting scratch scripts in `/tmp/`
Task session budgets	Global session caps: `max_tool_calls=15`, `max_turns=20`, `max_time_minutes=2.5`	Calling `submit_patch` before global budget exhaustion

Table 1: Separation between runtime-enforced constraints (`swegemma` / `google-adk`) and prompt-level directives.

4. Experimental Setup & Paired Evaluation Results

4.1 Hardware, Sampling Design, and Controlled Baselines

All experiments were executed on Kaggle's 4x NVIDIA L4 GPU environment (`machine_shape: "NvidiaL4"`, 96 GB total VRAM, `tensor_parallel_size=4`, `vLLM` serving `gemma-4-e4b-it-qat-w4a16-ct` with `max_model_len=32768`, `tool_call_parser="gemma4"`, `reasoning_parser="gemma4"`). Every configuration shares identical generation parameters (`temperature: 0.0`, `seed: 42`, `max_output_tokens: 8192`, `thinking_budget: 1536`, `include_thoughts: true`) and global task budgets (`max_tool_calls: 15`, `max_turns: 20`, `max_time_minutes: 2.5`, `timeout_seconds: 60`, `verification_timeout_seconds: 120`). The evaluated dataset ($n = 16$ unique tasks, 48 primary paired runs plus 16 rerun reproducibility runs = 64 total `swegemma` runs on 4x L4 GPUs) comprises two disjoint 8-task cohorts stratified across `fastapi/fastapi`, `Textualize/rich`, `psf/requests`, and `encode/httpx`: **Cohort 1** (first k_r tasks per repo in canonical `tasks.jsonl` order, $n = 8$) yielded 1/8 (12.5%) for `sfd` (`rich_4077`), 0/8 (0.0%) for `flat_starter`, and 0/8 (0.0%) for `flat_controlled`; **Cohort 2** (disjoint seeded-random stratified sample using `random.Random(42)`, $n = 8$) yielded 0/8 (0.0%) for `sfd`, 1/8 (12.5%) for `flat_starter` (`fastapi_9753`), and 0/8 (0.0%) for `flat_controlled`.

On each task, we evaluate three configurations back-to-back:

- **`flat_starter`**: Official competition starter kit (`sample_submission/agent.yaml` with `code_analyzer` `AgentTool` and `prompts/system.md`, adapted to `gemma-4-e4b-it-qat-w4a16-ct`).
- **`flat_controlled`**: Single `LlmAgent` matching `sfd`'s 9 tools, combined 3-phase prompt text, and sampling parameters inside a single continuous context (`include_contents: "default"`), differing from `sfd` in stage-scoped instruction/tool partitioning and `SequentialAgent` context resets.
- **`sfd`**: Our 3-stage `SequentialAgent` pipeline.

4.2 Aggregate Resolution, Patch Failure Split, and Context Metrics

Table 2 reports the aggregate metrics computed by `scripts/aggregate.py` from the ATIF v1.7 traces (`traces/`). To separate budget exhaustion from incorrect code edits, the **Fail: No Patch / Wrong Patch** column explicitly splits unresolved tasks into those that produced no patch due to time/tool budget exhaustion (`timeout_or_budget_no_patch`) versus those that generated a non-empty patch that failed Phase 2 `pytest` verification (`test_verification_failed`). **Mean Peak P_t** is the mean across tasks of each task's maximum single-call `prompt_tokens` ($\max_t P_t$), while **Max Peak P_t** is the maximum single-call `prompt_tokens` across all calls and tasks.

Configuration	Resolved	95% Wilson CI	Non-Empty Patch	Fail: No / Wrong Patch	Mean Peak P_t	Max Peak P_t	Mean Cum. Tokens	Mean Tools	Mean Time
Official Starter Kit (`agent_tool` pattern)	1/16 (6.2%)	[1.1%, 28.3%]	13/16 (81.2%)	3 / 12	13029.3	15281	103350.1	9.9	97.3s
Controlled Single-Agent (`LlmAgent`)	0/16 (0.0%)	[0.0%, 19.4%]	14/16 (87.5%)	2 / 14	13550.8	16154	119664.4	10.5	92.2s
State-Filtered Deliberation (`SequentialAgent`)	1/16 (6.2%)	[1.1%, 28.3%]	14/16 (87.5%)	2 / 13	11291.9	16146	88909.5	11.2	89.4s

Table 2: Paired evaluation results on `gemma-4-e4b-it-qat-w4a16-ct` ($n = 16$ tasks on 4x NVIDIA L4 GPUs). **Mean Tools** reports budget-counted tool calls (`session.state["tool_calls_used"]`).

Primary Resolution Tests & Exploratory Post-Hoc Paired Context/Cost Comparisons: While paired resolution tests (`resolved`) are our primary outcome, the paired sign tests and bootstrap CIs on peak prompt tokens, cumulative total tokens, and wall-clock duration reported below are **exploratory post-hoc analyses** (not pre-registered or adjusted for multiple comparisons; note that the `sfd` vs. `flat_controlled` sign test $p = 0.0213$ remains < 0.05 under a two-baseline Bonferroni correction):

- **`sfd` vs. `flat_starter`:** Resolution difference $\Delta = -0.00\%$, 95% paired bootstrap CI $[-18.75\%, +18.75\%]$, discordant pairs $(b = 1, c = 1)$, exact two-sided McNemar $p = 1.0000$. Exploratory paired peak prompt difference: -1737.4 tokens (-13.3%, 95% paired bootstrap CI $[-3768.0, +68.7]$), with `sfd` achieving a lower peak P_t on **11/16 tasks** (sign test $p = 0.2101$). Exploratory paired cumulative total token difference: -14440.6 tokens (-14.0%, 95% CI $[-33916.2, +4032.5]$, lower on 11/16 tasks, sign test $p = 0.2101$); wall-clock time difference: -7.97s (95% CI $[-26.38, +8.66]$s, faster on 8/16 tasks, sign test $p = 1.0000$).
- **`sfd` vs. `flat_controlled`:** Resolution difference $\Delta = +6.25\%$, 95% paired bootstrap CI $[-0.00\%, +18.75\%]$, discordant pairs $(b = 1, c = 0)$, exact two-sided McNemar $p = 1.0000$. Exploratory paired peak prompt difference: $\mathbf{-2258.8}$ **tokens** (-16.7%, 95% paired bootstrap CI $[\mathbf{-3734.9, -789.6}]$, excluding zero), with `sfd` achieving a lower peak P_t on **13/16 tasks** vs. 3/16 (exact two-sided sign test $\boldsymbol{p = 0.0213}$). Exploratory paired cumulative total token difference: $\mathbf{-30754.9}$ **tokens** (-25.7%, 95% paired bootstrap CI $[\mathbf{-52067.0, -10336.1}]$, excluding zero), with `sfd` using fewer cumulative tokens on **13/16 tasks** (sign test $\boldsymbol{p = 0.0213}$); wall-clock time difference: -2.82s (95% CI $[-19.46, +13.52]$s, faster on 7/16 tasks, sign test $p = 0.8036$).
- **Rerun Reproducibility under `vLLM` Tensor Parallelism (`tp=4, temperature=0.0, seed=42`):** Rerunning `sfd` ($n = 8$) and `flat_controlled` ($n = 8$) on Cohort 1 (`traces/rerun_results/`) produced 8/8 (100.0%) and 7/8 (87.5%) `resolved` agreement (`6/8` and `8/8` patch-presence agreement), with a mean absolute peak prompt drift of 2427.2 tokens (max 5292) for `sfd` and 1797.4 tokens (max 4457) for `flat_controlled`, consistent with run-to-run nondeterminism in parallel `vLLM`/ADK execution (`tp=4`). Crucially, while `sfd_rerun` again resolved `rich_4077` (1/8), `flat_controlled_rerun` also generated `self.__file.isatty()` and resolved `rich_4077` (1/8). Combined with `6/8` SFD patch-presence agreement across reruns, this shows that single-run resolution counts (`0/16` vs. `1/16`) at $n = 16$ are noisy and that the 1-task difference on `rich_4077` in Run 1 is within run-to-run execution variance.

Stage-Entry Context Reset vs. Trajectory Peak Context: The trace data demonstrate a clear distinction between **stage-entry** prompt length, **typical** per-task peak prompt length, and **upper-tail** trajectory peak prompt length. Across the `sfd` runs ($n = 16$), `TriageAgent` (which has `include_contents: "default"` and is never reset) reaches a mean peak prompt of **10922.1 tokens** and a maximum of **16146 tokens** (with the six highest `TriageAgent` peaks reaching `16,146` on `rich_4076`, `15,961` on `fastapi_14099`, `15,605` on `fastapi_14459`, `14,528` on `rich_3944`, `14,454` on `fastapi_15589`, and `14,446` on `requests_7502`). When `SequentialAgent` transitions to `PatchAgent` (`include_contents: "none"`), the entry prompt on `PatchAgent`'s first LLM call drops to a mean of **1500.9 tokens** (max **2126 tokens**)—a **−86.3% cross-boundary peak-to-entry reset** that holds by construction because `_get_current_turn_contents()` slices out prior tool history and `events[0]` and injects only the problem statement and `<localization_packet>`. Similarly, `VerifierAgent` entry resets to a mean of **1470.2 tokens** (max **2165 tokens**) after `PatchAgent` reaches a within-stage mean peak of **5355.9 tokens** (max **12680 tokens**). Because exploratory reading and code editing are split across a reset boundary rather than stacked into a single session, SFD achieves a lower per-task peak prompt length than the controlled single-agent baseline (`flat_controlled`) on 13/16 tasks (sign test $p = 0.0213$, mean reduction of 16.7%, or -2258.8 tokens) and lower than `flat_starter` on 11/16 tasks (mean reduction of 13.3%, or -1737.4 tokens). However, SFD does **not** bound upper-tail trajectory peaks when `TriageAgent` itself loops until budget exhaustion: SFD's maximum peak (16146) is virtually identical to `flat_starter` (15281) and `flat_controlled` (16154), and on tasks such as `requests_7502` (`14,446` vs. `10,436` / `11,238`) and `rich_4076` (`16,146` vs. `12,860` / `14,849`), SFD peaked higher than both baselines.

5. Per-Task Trace Analysis, Hypothesis Tests & Failure Audit

Task ID	Repository	`flat_starter` (Peak P_t)	`flat_controlled` (Peak P_t)	`sfd` (Peak P_t)
fastapi_14099	fastapi/fastapi	Fail (test verification failed) (15281)	Fail (timeout or budget no patch) (13933)	Fail (test verification failed) (15961)
fastapi_14361	fastapi/fastapi	Fail (test verification failed) (10191)	Fail (test verification failed) (15010)	Fail (test verification failed) (9831)
fastapi_14459	fastapi/fastapi	Fail (test verification failed) (14820)	Fail (timeout or budget no patch) (16154)	Fail (test verification failed) (15605)
fastapi_15588	fastapi/fastapi	Fail (test verification failed) (13083)	Fail (test verification failed) (10785)	Fail (test verification failed) (7291)
fastapi_15589	fastapi/fastapi	Fail (test verification failed) (14711)	Fail (test verification failed) (14912)	Fail (timeout or budget no patch) (14454)
fastapi_15661	fastapi/fastapi	Fail (test verification failed) (11099)	Fail (test verification failed) (14445)	Fail (test verification failed) (7180)
fastapi_9753	fastapi/fastapi	Pass (11047)	Fail (test verification failed) (11743)	Fail (test verification failed) (6469)
httpx_3672	encode/httpx	Fail (test verification failed) (14736)	Fail (test verification failed) (13010)	Fail (test verification failed) (8140)
requests_7427	psf/requests	Fail (test verification failed) (9747)	Fail (test verification failed) (15387)	Fail (test verification failed) (13362)
requests_7502	psf/requests	Fail (test verification failed) (10436)	Fail (test verification failed) (11238)	Fail (test verification failed) (14446)
requests_7505	psf/requests	Fail (test verification failed) (10800)	Fail (test verification failed) (12318)	Fail (test verification failed) (10659)
rich_3043	Textualize/rich	Fail (test verification failed) (14789)	Fail (test verification failed) (14820)	Fail (timeout or budget no patch) (14384)
rich_3944	Textualize/rich	Fail (timeout or budget no patch) (15175)	Fail (test verification failed) (15031)	Fail (test verification failed) (14528)
rich_4070	Textualize/rich	Fail (timeout or budget no patch) (13298)	Fail (test verification failed) (14826)	Fail (test verification failed) (8499)
rich_4076	Textualize/rich	Fail (test verification failed) (14577)	Fail (test verification failed) (15083)	Fail (test verification failed) (16146)
rich_4077	Textualize/rich	Fail (timeout or budget no patch) (14679)	Fail (test verification failed) (8117)	Pass (3716)

Table 3: Task-by-task paired outcomes and single-call peak prompt tokens ($\max_t P_t$) on `gemma-4-e4b-it-qat-w4a16-ct`.

Task ID	Triage Entry	Triage Peak	Patch Entry (Reset vs. Peak)	Patch Peak	Verifier Entry	Verifier Peak
fastapi_14099	2634	15961	1746 (-89.1%)	4145	1420	6860
fastapi_14361	2477	9831	1192 (-87.9%)	6365	1260	2117
fastapi_14459	2773	15605	1804 (-88.4%)	3633	1562	2496
fastapi_15588	3373	4979	1877 (-62.3%)	7291	1818	2333
fastapi_15589	3083	14454	1534 (-89.4%)	12680	1588	1588
fastapi_15661	2993	3873	1505 (-61.1%)	7180	1557	4875
fastapi_9753	2763	6469	1881 (-70.9%)	3030	1652	4976
httpx_3672	2116	7842	1261 (-83.9%)	5007	1304	8140
requests_7427	2072	13362	1326 (-90.1%)	5891	1347	1850
requests_7502	1914	14446	1480 (-89.8%)	2714	1427	3042
requests_7505	2102	10659	1293 (-87.9%)	2784	1362	1812
rich_3043	2618	14384	1388 (-90.4%)	7650	1285	2014
rich_3944	2892	14528	1187 (-91.8%)	4323	1255	2141
rich_4070	4829	8499	2126 (-75.0%)	5380	2165	7542
rich_4076	2901	16146	1212 (-92.5%)	4816	1268	1887
rich_4077	2903	3716	1203 (-67.6%)	2805	1254	1970

Table 4: Measured prompt token counts (P_t) at stage entry and stage peak across `sfd` runs (`traces/`). Note that **Patch Entry (Reset vs. Peak)** compares `PatchAgent`'s first call against `TriageAgent`'s peak call.

5.1 Testing Hypothesis 1: Observational Error Rates by Prompt-Length Bucket

To test whether longer prompts correlate with higher tool error rates in our recorded trajectories, Table 5a groups all LLM calls and tool invocations across the primary configurations (`flat_starter`, `flat_controlled`, `sfd`, 48 runs across $n = 16$ tasks) into four prompt-length buckets ($P_t < 4,000$; $4,000$–$7,999$; $8,000$–$11,999$; and $\geq 12,000$ tokens). Because global budget exhaustion (`BudgetExceeded` when `tool_calls_used == 15`) occurs late in trajectories by definition, Table 5a explicitly separates genuine non-budget tool errors (`FileEditError`, `CommandError`, `FileReadError`) from `BudgetExceeded` events, and Table 5b stratifies `edit_file` `FileEditError` rates by configuration.

Bucket (P_t)	LLM Calls	Tool Calls	Non-Budget Tool Err	Budget Exceeded	`edit_file` `FileEditError`	`edit_file` BudgetErr	Dup `read_file`
$<$ 4,000	155	138	18/138 (13.0%)	1/138	4/20 (20.0%)	0/20	6/50 (12.0%)
4,000 - 7,999	220	193	30/193 (15.5%)	3/193	12/38 (31.6%)	1/38	3/102 (2.9%)
8,000 - 11,999	183	160	22/160 (13.8%)	4/160	11/30 (36.7%)	2/30	6/54 (11.1%)
$\geq$ 12,000	85	72	13/72 (18.1%)	1/72	7/17 (41.2%)	0/17	5/25 (20.0%)

Table 5: Table 5a: Pooled observational non-budget tool error, `BudgetExceeded`, `edit_file` `FileEditError`, and duplicate `read_file` rates by per-call prompt-length bucket (P_t).

Configuration (`edit_file` `FileEditError`)	$< 4,000$	$4,000$–$7,999$	$8,000$–$11,999$	$\geq 12,000$
flat_starter	0/0 (0.0%)	7/15 (46.7%)	6/12 (50.0%)	3/7 (42.9%)
flat_controlled	0/0 (0.0%)	0/10 (0.0%)	5/17 (29.4%)	4/10 (40.0%)
sfd	4/20 (20.0%)	5/13 (38.5%)	0/1 (0.0%)	0/0 (0.0%)

Table 6: Table 5b: Per-configuration stratification of `edit_file` `FileEditError` rates across prompt-length buckets (P_t). Numerators exclude `edit_file` `BudgetExceeded` rejections (`1` in `flat_controlled` at $8,000$–$11,999$, `1` in `sfd` at $4,000$–$7,999$, and `1` in `sfd` at $8,000$–$11,999$, shown in Table 5a).

When `BudgetExceeded` rejections are separated from genuine tool errors, Hypothesis 1 is **inconclusive** in these traces: overall non-budget tool error rates remain flat across the first three buckets (13.0% at $P_t < 4,000$, 15.5% at $4,000$–$7,999$, and 13.8% at $8,000$–$11,999$) before rising slightly to 18.1% (13/72) at $P_t \geq 12,000$, while duplicate `read_file` rates are non-monotonic (12.0%, 2.9%, 11.1%, and 20.0%). Furthermore, as Table 5b shows, the pooled increase in `edit_file` `FileEditError` rates from 20.0% (4/20) at $P_t < 4,000$ to 41.2% (7/17) at $P_t \geq 12,000$ is confounded by stage and configu-

ration: every < 4,000 edit_file call comes from sfd (PatchAgent immediately after the 1.5k stage-entry reset), whereas within flat_starter (46.7%, 50.0%, 42.9%) and flat_controlled (0.0%, 29.4%, 40.0%), edit_file sample sizes per bucket are small (7–17 calls) and multiple calls come from the same trajectory.

5.2 Testing Hypothesis 2: Descriptive Reconciliation of code_analyzer_agent Calls in flat_starter

Inspection of the flat_starter ATIF v1.7 traces provides a descriptive accounting of code_analyzer_agent usage in this 16-task sample and reconciles the budget-counted tool metric in Table 2 with the raw ATIF trace event count:

- In swegemma, session.state["tool_calls_used"] (reported in Table 2 as **Mean Tools** = 9.9, or **158 budget-counted calls** across 16 tasks, $\frac{158}{16} = 9.875$) increments on workspace, graph, and shell tools (read_file, edit_file, write_file, run_command, search_similar_code, get_code_neighbors, get_code_subgraph), including those executed inside the child code_analyzer_agent, but does **not** increment on free submit_patch calls (13 calls) or on the parent AgentTool wrapper call code_analyzer_agent(request=...) itself (4 calls).
- Specifically, the **176 total ATIF trace tool_calls events** in flat_starter decompose into: **145 root-agent workspace/graph/shell calls + 14 child code_analyzer_agent calls** (4 on requests_7505, 3 on rich_4077, 7 on requests_7427, totaling 159 attempted budget-applicable calls, of which **158** incremented tool_calls_used and 1 call on fastapi_14099 was rejected after hitting the max_tool_calls=15 cap), plus **13 free submit_patch calls + 4 root code_analyzer_agent delegation calls** = **176 total trace tool events**.
- Out of 149 non-submit_patch tool decisions made by the root swe_baseline_agent in this 16-task sample (145 direct workspace/graph/shell calls vs. 4 code_analyzer_agent delegations), swe_baseline_agent delegated to code_analyzer_agent **4/149 times (2.7%)** (4/176 = 2.3% of all trace tool events), bypassing code_analyzer_agent on **13/16 tasks (81.2%)**.

5.3 Testing Hypothesis 3: Case Studies (rich_4077 & fastapi_9753), Ground-Truth Localization Taxonomy, and Audit of Unresolved SFD Tasks

Why sfd resolved rich_4077 while flat_starter and flat_controlled (Run 1) failed, and why flat_starter resolved fastapi_9753:

- On rich_4077 (Cohort 1, adding isatty() to FileProxy in rich/file_proxy.py): flat_starter spent all 15 budget-counted tool calls running commands and reading files (14,679 peak prompt tokens) without ever calling edit_file (0 patch chars). flat_controlled (Run 1) localized rich/file_proxy.py in 6 calls at its lowest peak prompt (8,117 tokens), but wrote import os and return os.isatty(self.__file.fileno()), which fails Phase 2 pytest verification (test_exit_code=1) because the wrapped mock file object in tests/test_file_proxy.py implements isatty() without exposing an integer fileno(). In sfd, TriageAgent read rich/file_proxy.py (3,716 peak prompt tokens) and wrote <localization_packet> explicitly specifying def isatty(self) -> bool: return self.__file.isatty(); PatchAgent entered at 1,203 tokens and applied that exact 3-line method (resolved=True). Moreover, in our rerun reproducibility study (traces/rerun_results/), both sfd_rerun (resolved=True) and flat_controlled_rerun (resolved=True) generated self.__file.isatty(), confirming that rich_4077 does not fail due to context overflow in single-agent mode.
- On fastapi_9753 (Cohort 2, supporting functools.partial in FastAPI dependencies): flat_starter read fastapi/dependencies/models.py directly, edited Dependant._unwrapped_call at step 6 (9,595 prompt tokens on the edit_file call, later reaching its trajectory peak of 11,047 prompt tokens at step 10 on submit_patch), and unwrapped isinstance(self.call, functools.partial) via return self.call.func (resolved=True). By contrast, sfd's TriageAgent (6,469 peak prompt tokens) localized fastapi/dependencies/utils.py instead of models.py, and PatchAgent merely added an unused from functools import partial import (test_verification_failed, Phase 2 test_exit_code=1). This illustrates a key trade-off of strict stage isolation: when TriageAgent localizes the wrong file (utils.py vs. models.py), PatchAgent (include_contents: "none") starts with only <localization_packet> and does not re-explore neighboring modules.

Phase 2 Exit-Code Breakdown and Ground-Truth Localization Taxonomy (tasks.jsonl): Cross-referencing the 16 sfd runs against the Stage 1–3 ATIF traces (traces/) and the ground-truth patch and test_patch specifications in tasks.jsonl yields an exact, non-speculative taxonomy of outcomes:

- **By Phase 2 Verification Exit Code (15 unresolved sfd runs): 5 tasks** exited with test_exit_code = 2 (pytest collection/import error prior to test execution): fastapi_14099 (where PatchAgent left a detached else: block at fastapi/routing.py:259, raising SyntaxError during collection), fastapi_15661 (where the reference patch creates a new module scripts/prepare_release.py imported by test_patch, whereas SFD edited scripts/add_latest_release_date.py, raising ModuleNotFoundError), and fastapi_14361, fastapi_14459, and httpx_3672 (where the offline sandbox venv lacked inline_snapshot or resolved import httpx to /usr/local/lib/python3.12/dist-packages/httpx across all 3 configurations, alongside wrong or incomplete file localization by SFD). **8 tasks** generated a non-empty patch that ran under pytest and exited with test_exit_code = 1 (test assertion failure: fastapi_15588, fastapi_9753, requests_7427, requests_7502, requests_7505, rich_3944, rich_4070, rich_4076). Finally, **2 tasks** produced a 0-byte patch due to tool or budget exhaustion (fastapi_15589, test_exit_code = -1; and rich_3043, test_exit_code = 1 after 4 FileEditError calls).
- **By Ground-Truth File & Symbol Localization (16 total sfd runs vs. tasks.jsonl reference patches):** SFD localized the **exact single reference file** on **4/16 tasks** (rich_4077 [resolved], fastapi_15588 [wrong ValueError message string

in `fastapi/sse.py`], `fastapi_15589` [localized `fastapi/dependencies/utils.py` but hit `BudgetExceeded` before editing], and `rich_3043` [localized `CONSOLE_HTML_FORMAT` in `rich/_export_format.py` but failed 4 `edit_file` calls]); achieved **partial localization on multi-file or multi-site reference patches** on **6/16 tasks** (`fastapi_14099` [1/3 files + `SyntaxError`], `fastapi_14459` [1/3 files], `httpx_3672` [1/7 files], `requests_7502` [1/2 sites in `src/requests/models.py`], `requests_7505` [1/2 files, omitting `src/requests/_types.py`], and `rich_4070` [1/9 files]); and localized the **wrong file or function** on **6/16 tasks** (`fastapi_14361`, `fastapi_15661`, `fastapi_9753`, `requests_7427`, `rich_3944`, `rich_4076`).

Task ID	Submitted SFD Patch	Failure Bucket (Exit)	Trace-Recorded Stage 1–3 Observation (`traces/`)	Phase 2 Outcome & Reference Comparison (`tasks.jsonl`)
`fastapi\_14099`	`fastapi/routing.py` (588 chars)	`test_verification_failed` (exit=2)	`P_t` peaked at 15,961 in `TriageAgent` (6,860 in `VerifierAgent`). `PatchAgent` inserted `for dep in dependencies: file_stack.callback(dep())` in `fastapi/routing.py`, leaving a detached `else:` at line 259; Stage 3 `run_command` Step 18 failed `pytest` collection (`SyntaxError: invalid syntax`, `exit_code=2`).	Phase 2 `pytest` exited with `test_exit_code=2` (collection error from `SyntaxError` at `fastapi/routing.py:259`). Reference patch in `tasks.jsonl` modifies `fastapi/applications.py`, `fastapi/middleware/asyncexitstack.py`, and `fastapi/routing.py`.
`fastapi\_14361`	`fastapi/openapi/models.py` (383 chars)	`test_verification_failed` (exit=2)	`PatchAgent` changed `Reference.ref` in `fastapi/openapi/models.py` to `Optional[str] = Field(default=None, alias='$ref');` Stage 3 `run_command` Step 14 ran `pytest tests/test_openapi_schema.py` (`exit_code=4`, file not found).	Reference patch modifies `_remap_definitions_a` in `fastapi/_compat/v2.py` (wrong file localized). Also, `test_patch` (`tests/test_schema_compat_pydanti` imports `inline_snapshot`, which is absent in the offline sandbox venv (`ModuleNotFoundError` at collection, `test_exit_code=2` across all 3 configs; seen in `flat_starter` Step 11).
`fastapi\_14459`	`fastapi/dependencies/models.py` (783 chars)	`test_verification_failed` (exit=2)	`PatchAgent` replaced ordered list deduplication in `Dependant.oauth_scopes` in `fastapi/dependencies/models.py` with `set(self.parent_oauth_scopes) | set(self.own_oauth_scopes);` Stage 3 `run_command` Step 18 ran `pytest tests/test_dependencies.py` (`exit_code=4`, file not found).	Reference patch modifies 3 files (`fastapi/dependencies/models.py`, `fastapi/dependencies/utils.py`, `fastapi/openapi/utils.py`; SFD edited 1/3 files). Also, `test_patch` imports `inline_snapshot`, causing `pytest` collection to exit with `test_exit_code=2`.
`fastapi\_15588`	`fastapi/sse.py` (1008 chars)	`test_verification_failed` (exit=1)	`PatchAgent` replaced `_check_id_no_null` with `_check_sse_string` (`\0`, `\n`, `\r`) in `fastapi/sse.py`, raising `ValueError('SSE field cannot contain null, newline, or carriage return characters');` existing `tests/test_sse.py` passed in Stage 3 Step 15 (18 passed).	Phase 2 `pytest` exited with `test_exit_code=1` (assertion failure): `test_patch` (`tests/test_sse.py`) asserts `pytest.raises(ValueError, match=f"SSE '{field_name}' must be a single line")`, which did not match SFD's error message string.
`fastapi\_15589`	None (0 chars)	`timeout_or_budget` (exit=-1)	`TriageAgent` made 10 read-only calls (`P_t` peaked at 14,454); `PatchAgent` entered at 1,534 tokens, read 6 slices of `fastapi/dependencies/utils.py`, and hit `BudgetExceeded` (`max_tool_calls=15`) on Step 20 `edit_file` (0 patch chars).	No patch submitted (`test_exit_code=-1`). Reference patch adds `processed_keys.add(get_validation` at line 829 of `fastapi/dependencies/utils.py` (correct file localized by `TriageAgent`, but budget exhausted before edit).
`fastapi\_15661`	`scripts/add_latest_release_date.py` (660 chars)	`test_verification_failed` (exit=2)	`PatchAgent` rewrote `RELEASE_HEADER_PATTERN` in `scripts/add_latest_release_date.py`; Stage 3 `run_command` Steps 14 & 17 failed with `CommandError: No release header found`	Reference patch creates a new 216-line CLI module `scripts/prepare_release.py`, and `test_patch` (`tests/test_prepare_release.py`) imports `from scripts.prepare_release`
`fastapi\_15687`	`fastapi/dependencies/utils.py` (467 chars)	`test_verification_failed` (exit=2)	`P_t` peaked at 16,467 in `TriageAgent` (6,860 in `VerifierAgent`). `PatchAgent` entered at 1,534 tokens, read 6 slices of `fastapi/dependencies/utils.py`, and hit `BudgetExceeded` (`max_tool_calls=15`) on Step 20 `edit_file` (0 patch chars).	Phase 2 `pytest` exited with `test_exit_code=2` (collection error from `ImportError` at `fastapi/dependencies/utils.py:829`). Reference patch in `tasks.jsonl` modifies `fastapi/dependencies/utils.py`, `fastapi/routing.py`, and `fastapi/sse.py`.

Table 7. Complete task-by-task audit of all 15 unresolved Pro tasks separating Stage 1–3 observations (`traces/`) from Phase 2 verification outcomes (1 = assertion failure, 2 = collection/import error, -1 = no patch) and comparing SFD patches against reference patches (`tasks.jsonl`).

6. Related Work

Staged vs. Monolithic SWE Agents. SWE-agent ((Yang et al., 2024)) and OpenHands ((Wang et al., 2024)) equip an LLM with an interactive bash/file environment inside a continuous trajectory, whereas Agentless ((Xia et al., 2024)) uses a fixed pipeline of localization and repair without autonomous tool loops. SFD combines tool-using ADK sub-agents with stage-boundary context slicing.

Observation Masking vs. LLM Summarization. Lindenbauer et al. ((Lindenbauer et al., 2025)) (**"The Complexity Trap"**, JetBrains Research & TU Munich) show that simple observation masking of older tool outputs halves token costs while matching or outperforming LLM-based trajectory summarization on SWE-bench Verified. SFD's `include_contents: "none"` transition acts as a coarse stage-boundary observation mask.

7. Limitations & Threats to Validity

1. **Sample Size, Repository Mix, and Statistical Power:** At $n = 16$, `sfd` (1/16 = 6.2%) and `flat_starter` (1/16 = 6.2%) tie in resolution rate (McNemar $p = 1.0000$), and the 1-task difference over `flat_controlled` (0/16 = 0.0%) yields McNemar $p = 1.0000$ and disappears on Cohort 1 rerun (`1/8` each). Detecting a 10 percentage point paired difference (e.g., 15% vs. 5% with 12% discordant pairs) at 80% power ($\alpha = 0.05$, two-sided McNemar test) requires $N \approx 115$–130 paired tasks (essentially the entire 129-task benchmark). In addition, because our stratified cohorts guarantee representation of all four repositories, the 16-task sample (`7 fastapi` [43.8%], `5 rich` [31.2%], `3 requests` [18.8%], `1 httpx` [6.2%]) slightly over-represents `requests` and `httpx` relative to the full 129-task benchmark (`67 fastapi` [51.9%], `48 rich` [37.2%], `13 requests` [10.1%], `1 httpx` [0.8%]).
2. **Confounding of Budget Limits (`max_tool_calls=15`, `max_time_minutes=2.5`) with Context Effects:** As shown in Table 2 and Table 6, 3/16 `flat_starter` runs (`rich_3944`, `rich_4070`, `rich_4077`), 2/16 `flat_controlled` runs (`fastapi_14099`, `fastapi_14459`), and 2/16 `sfd` runs (`fastapi_15589`, `rich_3043`) failed to produce a patch (`timeout_or_budget_no_patch`). Without per-stage code caps on tool calls, global budget exhaustion confounds purely architectural comparisons.
3. **Propagation of Triage Localization Errors Across Isolated Stages:** As demonstrated by `fastapi_9753` (`flat_starter` resolved by editing `models.py`, whereas `sfd` localized `utils.py`) and `rich_4070`, when `TriageAgent` misidentifies the target file or omits a companion module in a multi-file bug, `PatchAgent` receives only `<localization_packet>` and cannot recover from the upstream localization error.

8. Conclusion & Reproducibility

We presented and empirically audited State-Filtered Deliberation (SFD) on `gemma-4-e4b-it-qat-w4a16-ct` across 64 real `swegemma` + `vLLM` execution runs ($n = 16$ paired tasks across 3 configurations = 48 primary runs, plus 16 rerun reproducibility runs) on 4x NVIDIA L4 GPUs. Our traces establish three findings in this 16-task sample: (1) in the official starter kit (`flat_starter`), the 4B model bypassed the discretionary `code_analyzer_agent` on 13/16 tasks (81.2%), invoking it on 4/149 (2.7%) of root non-`submit_patch` tool decisions; (2) configuring `include_contents: "none"` in a 3-stage `SequentialAgent` deterministically resets **stage-entry** prompt length (`PatchAgent` entry mean 1500.9 tokens vs. `TriageAgent` peak mean 10922.1 tokens, a −86.3% reset) and in exploratory post-hoc paired tests lowers per-task peak prompt length relative to `flat_controlled` on 13/16 tasks (sign test $p = 0.0213$, mean reduction of 16.7%, 95% CI $[-3734.9, -789.6]$) and cumulative total tokens by 25.7% (95% CI $[-52067.0, -10336.1]$, sign test $p = 0.0213$), without bounding upper-tail `TriageAgent` peaks (max 16146 tokens); and (3) end-to-end task resolution on 4-bit quantized E4B (1/16 = 6.2% for `sfd` and `flat_starter` vs. 0/16 = 0.0% for `flat_controlled`, McNemar $p = 1.0000$) shows a null overall resolution difference, with `tasks.jsonl` reference comparisons showing `4/16` exact single-file localizations, `6/16` partial multi-file/multi-site localizations, and `6/16` wrong file/function localizations. All 64 execution traces (`traces/`), leaderboard snapshots (`results/`), and scripts (`scripts/`) are publicly available at `https://github.com/KarandeepSinghSodhi/gemma-sfd-agent`.

References

Gemma Team, Google DeepMind. (2026). Gemma 4 Technical Report. *Arxiv Preprint Arxiv:2607.02770*.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. *International Conference on Learning Representations (ICLR)*.

Lindenbauer, T., Slinko, I., Felder, L., Bogomolov, E., & Zharov, Y. (2025). The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization for Agent Context Management. *Arxiv Preprint Arxiv:2508.21433*.

Wang, X., Li, B., Song, Y., Frank, F. F., & others. (2024). OpenHands: An Open Platform for AI Software Developers as Generalist Agents. *Arxiv Preprint Arxiv:2407.16741*.

Xia, C. S., Deng, Y., Dunn, S., & Zhang, L. (2024). Agentless: Demystifying LLM-based Software Engineering Agents. *Arxiv Preprint Arxiv:2407.01489*.

Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., & Press, O. (2024). SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *Arxiv Preprint Arxiv:2405.15793*.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations (ICLR)*.